

Distributed Agile, Agile Testing, and Technical Debt

Raja Bavani, Mindtree

// An interview with Johanna Rothman and Lisa Crispin aims to reveal perspectives on technical debt in agile teams with specific reference to agile testing and distributed teams. //



PRIMARILY FOUNDED AS methodologies for software projects executed at a single location, agile methodologies have shown promising results in multisite projects, with many adopters and practitioners spread out across the globe. Meanwhile, during the past decade, thought leaders and practitioners have recognized the role of agile testing or testing in agile projects and have articulated the significance of related engineering practices such as test-driven development, behavior-driven development, and test automation.

Agile testing in distributed teams is driven by how the teams are structured at each location. For development projects, a logical approach is to have

two or more distributed teams focus on independent modules or functionalities. In this case, teams in each location are cross-functional, with one or more test engineers focused on testing activities. This structure provides for optimal communication as well as dependencies across sites. With maintenance projects, it's beneficial to consider independent testing teams in addition to cross-functional ones. This is because cross-functional teams focus on the current iteration, whereas independent testing teams focus on the current release and contribute to areas such as security testing, performance testing, maintenance, and enhancing automated test suites.

In this article, I present an interview I conducted with Johanna Rothman and Lisa Crispin in March 2012, when I met with them at Belgium Testing Days 2012. The goal of my interview was to understand their perspectives on technical debt in agile teams with specific reference to agile testing and distributed teams. I also share my thoughts on the need for geographically distributed teams to become more aware and aligned to effectively manage technical debt.

The Interview

Bavani: “Technical debt” is a common term used by agile teams. What are the implications of technical debt on agile testing teams? What does it mean, or how does it matter, to agile testing teams?

Rothman: I don't really think of testing teams in agile. I think of project teams in agile. When many project teams transition to agile, they accumulate technical testing debt. They have lots of manual tests, but not too many automated tests. So, if you have a lot of manual tests, how do you keep up with the demand of the iteration, or flow, if you're using Kanban? It's really an issue of making it to the end of the iteration and getting it done. Then you have this product owner saying, “I want a feature! I want a feature!”—I wrote a related blog posting called “Do You Have Feature-itis?”¹

The issue is that developers are new to agile. They don't realize that testers have to keep up. They don't realize that their job is to help testers. And the product owner doesn't realize that he or she can't get all these features right now. Also, the product owner doesn't want to put technical debt into the iteration. You have all these people who are new to agile, and none of them realize that you have to rank technical



debt. Everyone's floundering. In the meantime, how do you get the product out the door?

First, don't think about agile testing teams; think about project teams that are new to agile. Second, think about how we get technical debt—whether it's architecture debt, testing debt, unit testing debt, or system testing debt. It's a hard thing. The faster you rank it, the easier it is to get the product out the door. The more technical debt you rank in the earlier iterations, the fewer features you get. You have this push and pull about how many features you get versus how much technical debt you pay off. There's no one right answer.

Bavani: How do you relate technical debt, agile, and agile testing teams?

Crispin: Technical debt is central to the definition of agile. Elizabeth Hendrickson calls it the acid test for agile: if a team is agile, it delivers business value frequently—at least every one to four weeks, depending on the customer's preference—adapting to the changing needs of business at a sustainable pace. A sustainable pace is the key. This is where technical debt comes in. You can't maintain a sustainable pace if you have technical debt.

With reference to agile testing teams, I personally prefer to have cross-functional integrated teams, but there could be good reasons to have separate testing teams—for example, companies that produce embedded software might need separate teams for testing.

Bavani: When the product is very critical and the acceptance criteria are stringent, we set up an independent testing team. There's also a development team that does coding, unit testing, and acceptance testing. The testing team performs functional and other types of tests. What



MEET THE EXPERTS

Johanna Rothman is an eminent author and speaker. She works with companies to improve how they manage their product development to maximize management and technical staff productivity and to improve product quality. She has authored several books, white papers, and articles. Visit her website at www.jrothman.com.

Lisa Crispin is an agile testing coach, author, and speaker. She specializes in showing testers and agile teams how testers can add value and how to guide development with business-facing tests. She has authored several books, white papers, and articles. Visit her website at www.lisacrispin.com.

are the repercussions of inadequate technical debt management on agile testing teams?

Rothman: That's really hard. That's where you have to find a way to partner with remote teams to start working together. I understand the challenges of distributed teams—I teach workshops on them, I've been writing about them, and I know that managers all over the world think that it's a perfectly fine thing to do. I think they're lunatics.

You have to start partnering with the people who are remote to you. The more critical the product, the more critical it is to partner testers with developers. You have to start thinking about how you'll define "done." Does it mean partially done, as in it's ready for testing, or fully done, as in it's ready to release? That's when Kanban is very helpful. It makes work in progress transparent. It allows you to ask, "Do I want acceptance criteria on what I turn into the testing queue?" Inside the iteration, if you have technical debt on work products transferred from the development team to testing, you have a situation that's not helping anybody. This is why I really like thinking about value streams. If you're working in iterations, fine. You can use a Kanban board and

start looking at the queues and value streams.

Bavani: Sometimes the management of technical debt doesn't happen in the right way. It becomes a catch-up game. When that happens, engineers who are responsible for testing suffer. What are the effects of this that you see?

Crispin: It affects everybody, because if we're not managing technical debt, we have code that isn't well designed, well written, easy to maintain, or supported by automated tests. So, if we want to change that code, it will take a long time for us to figure out what it does. If we don't have adequate test coverage, we'll have to spend a lot of time doing manual testing. Testing will become more and more of a bottleneck. This involves a lot of risks because there are so many unknowns. It will affect everybody on the team.

Bavani: Should agile testing teams be aware of technical debt and participate in discussions related to it? How can they contribute?

Rothman: I think it's worthwhile to talk about the definition of "done" and the steps the software goes through to

move from requirements all the way through to the final deployment and testing. Testing starts at different times depending on the kind of life cycle you choose. Are testers partially responsible for setting the acceptance criteria? If so, then they're also partially responsible for defining "done." That means that they're responsible from the very beginning.

Let's get testers involved from the very beginning. If you have an agile development team in one location and an agile testing team in another location, how do you get these two teams together to decide on the acceptance criteria? This is a big challenge, but it's possible to handle.

Bavani: Lisa, do you think testers can survive by standing alone and being ignorant of technical debt?

Crispin: I don't think they can stand alone. I really feel that everybody needs to take responsibility for quality and for keeping technical debt at a manageable level—that includes developers, testers, system administrators, business analysts, and database

I think the whole team has to make a meaningful commitment to do something about technical debt. They also need some management support to budget time to manage technical debt.

There are a number of reasons, in my opinion, for incurring technical debt: we get rushed, or we want to meet a business deadline. That's okay. The problem is that we don't go back and pay off that technical debt before the next deadline, so the debt never stops, it just gets worse.

Bavani: Test automation is one of the core focus areas of agile testing teams. How can they understand and manage technical debt related to automation design or scripting? Are they aware of technical debt?

Rothman: I think some testers are, but not enough of them. I was just working with a client a few weeks ago where the developers were using XP and the testers weren't inside the iteration. They were doing manual testing outside the iteration. I said to them, "You aren't really doing XP. You know that, right?" They said, "We're not?" I said, "If the

that particular instance, the fact is that testers were unaware of test automation and incapable of doing it. I told the company that if it doesn't work on test automation fast, it would require a lot of long-term effort to pay off that technical debt. It was creating an entire product with zero test automation!

Crispin: That's an excellent point. Automation test code is code. It's as valuable as production code, so all the same criteria apply to it. I have met many teams working in agile environments whose intentions were to automate all tests. That intention was good, but the teams weren't necessarily automating in a good way: they didn't pay attention to test design; their test scripts had lots of duplication; and they didn't extract duplicate code into reusable components. So they spend all of their time maintaining their regression tests. This defeats the purpose.

You need automated tests to free up your time to do more important things, such as manual exploratory testing. If you spend all of your time executing and maintaining your automated tests, you don't get any return on investment; it's a complete waste. We have to think about technical debt in our automated scripts, too. That's why I feel like people who have expertise in code design, such as the developers who write production code, also need to be involved in the design of automated test scripts. In my own team, developers have always been involved in automating the unit tests and API level tests. The GUI test automation is left to us as the testers, and we enjoy doing that automation because we all have programming backgrounds. We do a good job of it and get good returns on those tests. Even so, we still feel that if programmers were involved in helping us design the tests, we would spend less time in maintaining them. So, on my team, we are actually making an effort to get the

The whole team has to make a meaningful commitment to do something about technical debt.

administrators. Everybody involved in delivering software needs to be aware of it because technical debt can cross many areas. Consider a database team that isn't doing a good job of version control on database changes; there can be mistakes in the database, columns that aren't used anymore, or poorly named columns that cause confusion. All these contribute to technical debt. This can happen in any area. So,

developers are doing XP and testers are doing manual testing outside the iteration, you aren't doing XP. You don't have a releasable product at the end of the iteration." They said, "Oh ... okay."

I found that kind of distressing. They didn't realize that just having developers do XP wasn't XP. The testers don't know how to write automated tests. The developers don't even know what XP really is. I think, in

programmers more involved in helping us to design GUI tests.

There's definitely room for improvements in this area. I think what happens too often is that some team is tasked with, "Okay, you're the automation team—go and automate the tests," and they don't get the help they need, and they don't realize that they need help until they're in trouble. That's too late.

Bavani: In this context, what are the challenges you see in distributed teams? How can we address these challenges?

Rothman: One of the key things is that you get geographically distributed cross-functional teams together—literally and physically together in one location. There are smart people all over the world. To start distributed agile, try and bring everyone together for release planning at the very beginning of the project. Everyone knows this.

Managers will often say, "That's so expensive!" But if you really want a product that will release in your lifetime with as few defects as you probably want, you do it. You either pay for travel now or pay for defects later. It's impossible for us to trust each other if we've never met each other. Trust is the very first thing you have to create.

Let's say you and I meet each other. We work together for a few days. We have dinner together. You show me pictures of your family. You become a real person to me! And now, I know you. When I say to you, "That isn't acceptance criteria," or you say to me, "That isn't what 'done' means," we can fight it out and get it together because we understand each other. Now, you have a face, an attitude. I understand what you mean. When you send me an email, I can hear your voice in the email.

Without this initial meeting and working together, when you receive my email, you probably don't know when I'm having a bad day or that I have

150 other emails to read, so I briefly respond, "Okay." I mean, "Okay." I'm not mad at you, I'm just saying, "Okay." That's what we do in real life. The problem is that email divorces all the music from our words. This is the problem with geographically distributed teams. I truly advocate that people come together and build trust in distributed teams.

Crispin: I work with a distributed team on a small scale. We had one developer

two-week sprint every six months to manage our technical debt. In this sprint, we don't deliver any business value, we just pay off technical debt. It's too risky to do any major refactoring or upgrade to your tools or frameworks when you are trying to deliver business functionality. So, although we try to refactor all the time, we feel that we need that two-week sprint to keep our technical debt under control. This is magnified with distributed teams. They have to make it even more vis-

You either pay for travel now
or pay for defects later.

in India and the rest of us in Colorado. Even on such a small scale, you see problems with the time difference. If the developer in India checks in some code and the developers in Denver feel that the code isn't designed well, then either they have to change it or they have to ask him to change it. If they're going to ask him to change it, they have to wait another day. If you blow that up into global companies working in different areas with several teams around the globe, you just multiply that problem. So, I think you have to pay extra attention to making technical debt visible.

There are lots of tools available to quantify technical debt, such as continuous integration tools. You can set up factors and criteria in these tools to give you a monetary amount of your technical debt based on the code complexity, automated test coverage, and so on. This will let you ask, "What is our level of technical debt? Is it growing in such a way that it is impacting our velocity? Do we need to stop and spend some time on this?"

In my own team, we spend a

ible and make sure that they invest that time. It's an investment—it's going to pay off. By reducing technical debt, the velocity is going to improve. Their business can count on a continuous stream of value coming out of that development team.

The Implications

According to Jim Highsmith, technical debt, if not managed properly, can destroy predictability, which can in turn impact market capitalization.² Israel Gat, who has done significant work in quantifying and identifying the financial cost of technical debt, warns that neglected technical debt can lead to a broad spectrum of difficulties from collapsed road maps to an inability to respond to customer problems in a timely manner.³ (You can read some of my related blog posts at www.blogs.mindtree.com/author/raja-bavani.)

Martin Fowler conceptualized and divided technical debt into four quadrants.⁴ He said it's worth understanding the differences between prudent and reckless debt created because of

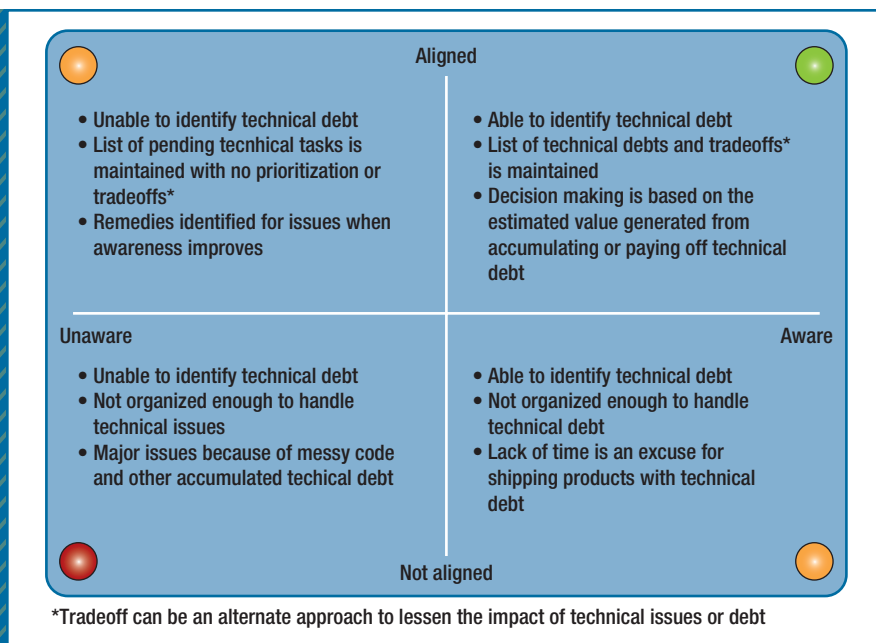


FIGURE 1. Technical debt quadrants. The four-quadrant model can help teams understand and manage technical debt better.

deliberate or inadvertent actions. His model motivated me to think further about the learning needs and alignment among distributed teams so that geographically distributed teams can become better at managing technical debt.

Learning and experiencing the various types of technical debt and technical debt management involves everyone in project teams. It's not sufficient for teams to be merely aware of what constitutes technical debt; they must be aligned to manage technical debt in order to deliver business value. This is complex and critical in projects executed across multiple sites. The four-quadrant model shown in Figure 1 can help teams understand this phenomenon and prompt them to manage it better.

Distributed agile teams are aware when team members have a common understanding of the meaning of technical debt and are mindful of it. These teams recognize technical debt either instantaneously or during reviews and retrospectives. Teams that are unaware

neither have a common understanding about the meaning of technical debt nor are they mindful of it.

Distributed teams are aligned when they have a common understanding about how to organize and manage technical debt, and they pay it off to increase business value delivered to stakeholders. Distributed teams aren't aligned when they lack a common understanding on how to organize and manage technical debt, become reactive in paying off technical debt, and eventually fail to deliver business value.

Aware and Aligned

Teams that are aware and aligned ensure positive results in managing technical debt. They can differentiate between trivial code quality issues and technical debt. They prioritize technical debt, consider tradeoffs (alternative approaches that will lessen the impact of the debt or to pay it off entirely), and make informed decisions while accumulating as well as paying off technical

debt. Even if one or more team members trigger technical debt by accident, the team recognizes it during reviews and retrospectives.

Unaware and Aligned

Teams that are unaware of the context or meaning of the term "technical debt" might consider trivial issues or technical tasks to be part of it. These teams lack the ability to distinguish between messy code and a design flaw. In this case, design tradeoffs won't be the only triggers to make additions to the list of items that constitute technical debt. These teams also don't know how to prioritize technical debt. When team members improve their awareness of technical debt, they'll start delivering value by paying off the technical debt.

Aware and Not Aligned

Simply being aware of technical debt doesn't necessarily result in business value. When agile teams don't have an organized approach for accumulating and managing technical debt, it's highly likely that they'll deliver messy and rigid software that's difficult to maintain.

These teams do know how and when they accumulate technical debt. However, because of their unorganized approach, they tend to blame a lack of time for releasing a product with technical debt, or they pay off technical debt without tradeoffs and don't deliver any business value. This is because when teams aren't aligned, there's no guarantee that they'll systematically pay off technical debt. This approach doesn't deliver value to project sponsors. In distributed agile projects, this situation might also lead to the blame game.

Unaware and Not Aligned

Teams in this quadrant pose high risks because they accumulate technical debt unknowingly and don't attempt to pay it off. Such teams encounter

major issues such as messy code or other forms of technical debt.

To manage technical debt, distributed agile teams need to be aware and aligned. They must track technical debt and know the value generated by paying it off. They must also include user stories related to paying down some of the technical debt in future iterations so that technical debt dwindles over a period of time.

In addition to understanding accumulating and paying off technical debt related to the application or product being developed or maintained, testers must be aware and aligned to manage the technical debt accumulated during the design, coding, and maintenance of test automation scripts. The significance of technical debt management directly affects the economics of software maintenance. Building and maintaining software systems under optimal costs is the key to success with current global economic trends. Rothman's and Crispin's perspectives emphasize the fact that technical debt is a significant concept for everyone in the software engineering community. With the adoption of distributed agile, internalizing a concept like technical debt becomes critical among global teams. 

References

1. J. Rothman, "Do You Have Feature-itis?," blog, 22 June 2011; www.jrothman.com/blog/mpd/2011/06/do-you-have-feature-itis.html.
2. J. Highsmith, "The Financial Implications of Technical Debt," blog, 19 Oct. 2010; <http://jimhighsmith.com/2010/10/19/the-financial-implications-of-technical-debt>.
3. I. Gat, "Technical Debt," *Cutter IT J.*, vol. 23, no. 10, 2010, pp. 3–6.
4. M. Fowler, "Technical Debt Quadrant," 14 Oct. 2009; www.martinfowler.com/bliki/TechnicalDebtQuadrant.html.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.



ABOUT THE AUTHOR



RAJA BAVANI is chief architect of Mindtree's Product Engineering Services (PES) and IT Services (ITS) groups. His research interests include code quality, distributed agile, customer value management, and software estimation. Bavani received a postgraduate diploma in computer science from Bharathidasan University, India. He's a member of IEEE and the IEEE Computer Society. Contact him at raja_bavani@mindtree.com.

IEEE CloudCom 2012

4th IEEE International Conference
on Cloud Computing Technology and Science

3-6 December 2012

Taipei, Taiwan

The "Cloud" is a natural evolution of distributed computing and the widespread adaption of virtualization and Service Oriented Architecture. In Cloud Computing, IT-related capabilities and resources are provided as services, via the Internet and on-demand, accessible without requiring detailed knowledge of the underlying technology. IEEE CloudCom 2012, steered by the Cloud Computing Association, brings together cloud computing researchers and related technologies.

Register today!

<http://2012.cloudcom.org/>



IEEE

IEEE  **computer society**